

Inclusión de ficheros

Contenido

Código Spaghetti

Inclusión de ficheros

Código Spaghetti

Como el propio nombre indica, el código spaghetti es un código desordenado y difícil de mantener, similar a un plato de espaguetis enredados. Este tipo de código suele surgir cuando no se siguen buenas prácticas de programación, como la modularización y la reutilización de código. A continuación, se muestra un ejemplo de código spaghetti en PHP:

```
<div class="stock-status">
<?php
    if ($stock > 0) {
        echo " <span>Disponible </span>";
    } else {
        echo " <span style='color:red;'>Agotado </span>";
    } ?>
</div>
```

Como podéis observar no es un código distinto del que hemos estado haciendo en clase, pero este tipo de programación es muy complicada de mantener. Para evitar que esto ocurra, es recomendable utilizar la inclusión de ficheros en PHP, lo que nos permite dividir el código en módulos reutilizables y más fáciles de gestionar. También facilita la colaboración entre varios desarrolladores, ya que cada uno puede trabajar en diferentes partes del proyecto sin interferir con el trabajo de los demás. En el futuro también usaremos frameworks que nos ayudarán a mantener un código limpio y organizado.

Inclusión de ficheros

Más contenido y explicaciones.

Función	Descripción	Caso de error	Ejemplo
include ''	Incluye el fichero en cada ocasión.	Da una advertencia, pero continua la ejecución.	include 'tu_archivo.php'
include_once ''	Incluye el fichero solo una vez.	Da una advertencia, pero continua la ejecución.	include_once 'tu_archivo.php'
require ''	Incluye el fichero en cada ocasión.	Para la ejecución (Error fatal).	require 'tu_archivo.php'
require_once ''	Incluye el fichero solo una vez.	Para la ejecución (Error fatal).	require_once 'tu_archivo.php'

Ejemplo de uso de include: Código html que se repite en todas las páginas, como el pie de página:

```
<footer style="font-family: Arial, sans-serif; background-color: #f8f9fa; padding: 30px 20px; border-top: 1px solid #dee2e6;">
    <div style="display: flex; flex-wrap: wrap; justify-content: center; align-items: center; max-width: 1200px; margin: 0 auto; text-align: center;">
        <div style="flex: 1 1 100%; margin-bottom: 20px;">
            <p style="margin: 5px 0; color: #555; font-size: 0.9em;">Este documento se ha realizado por Marín Capilla Herrero. Publicada bajo licencia</p>
            <p style="margin: 5px 0; color: #555; font-size: 0.9em;">Creative Commons Atribución/Reconocimiento-CompartirIgual 4.0 Internacional.</p>
            <p style="margin: 5px 0; color: #555; font-size: 0.9em;">Las modificaciones son permitidas siempre que sean mencionadas, la licencia no se puede modificar</p>
            <a href="https://creativecommons.org/licenses/by-sa/4.0/" target="_blank" rel="noopener noreferrer" style="display: inline-block; margin-top: 10px;">
                
            </a>
        </div>
        <div style="display: flex; flex-wrap: wrap; justify-content: center; align-items: center; gap: 15px;">
            
            
            
            
        </div>
    </div>
</footer>
```

Este código lo copiaríamos dentro de un fichero con nombre footer.php y luego se incluiría de la siguiente forma

```
<?php require_once 'footer.php'; ?>
```

¿Por qué *require* en lugar de *include*?

Un **footer** (pie de página) es una parte fundamental de la estructura y la interfaz de tu página web. Aunque el contenido principal pueda funcionar sin él, la página estaría visualmente rota o incompleta.

- **require:** Si el fichero `footer.php` no se encuentra (por un error en la ruta, porque se ha borrado, etc.), `require` lanzará un **error fatal** y detendrá la ejecución del script. Esto es bueno. Te obliga a darte cuenta inmediatamente de que algo va mal y a solucionarlo, en lugar de mostrar una página "rota" a los usuarios.
- **include:** Si usaras `include`, el script solo mostraría una **advertencia** y continuaría ejecutándose. El resultado sería una página sin pie de página, lo que es una mala experiencia para el usuario y un error que podría pasar desapercibido para ti durante más tiempo.

En resumen, para componentes esenciales de la plantilla (como la cabecera, el menú o el pie de página), es mejor ser estricto y forzar un error si no se encuentran.

¿Por qué añadir el sufijo `_once`?

require_once: Esta función garantiza que el fichero `footer.php` se incluya **una sola vez**, sin importar cuántas veces se llame a la instrucción en el flujo de tu código. En aplicaciones complejas, es posible que un fichero se intente incluir varias veces por accidente. Usar `_once` previene la duplicación de código (que podría romper el HTML) y ahorra recursos.

Conclusión

Para incluir el `footer.php`, `require_once` es la opción más robusta y segura. Te protege contra dos problemas comunes:

- **Ficheros no encontrados** (usando `require`).
- **Inclusiones duplicadas** (usando `_once`).

